BMC Bioinformatics

https://doi.org/10.1186/s12859-026-06546-5

Article in Press

SNPio: a Python interface for population genomic data processing

Received: 7 March 2026

Accepted: 24 June 2026

Published online: 03 July 2026

Cite this article as: Martin B.T., Monaco D.R., Sharabi N. *et al.* SNPio: a Python interface for population genomic data processing. *BMC Bioinformatics* (2026). https://doi.org/10.1186/s12859-026-06546-5

Bradley T. Martin, Domenico R. Monaco, Nadine Sharabi, Steven M. Mussmann & Tyler K. Chafin

SNPio: A Python interface for population genomic data processing

Bradley T. Martin[1],*, Domenico R. Monaco[1], Nadine Sharabi[1], Steven M. Mussmann[2], Tyler K. Chafin[3]

[1]Department of Biological Sciences, Seton Hall University, South Orange, NJ 07079, USA

[2]Abernathy Fish Technology Center, U.S. Fish and Wildlife Service, Longview, WA 98632, USA

[3]Department of Biological Sciences, University of Arkansas, Fayetteville, AR 72701, USA

*Corresponding author: bradley.martin@shu.edu

Abstract

Background: Population genomic workflows frequently rely on fragmented command-line utilities, custom conversion scripts, and programming language-specific environments, complicating computational reproducibility and obscuring data provenance. As analytical workflows become increasingly automated and computationally intensive, dependence on disparate preprocessing tools can introduce friction between raw genotype files, quality-control decisions, statistical analyses, and downstream workflows. We developed SNPio, a Python-native framework that consolidates single nucleotide polymorphism data parsing, filtering, visualization, numerical genotype encoding, and population genomic summary-statistic calculation within a unified software architecture.

Results: VCF file parsing and filtering benchmarks were compared against vcfR and SNPfiltR. SNPio demonstrated faster execution times but used more memory than its R-based comparators, reflecting SNPio's retention of genotype arrays, metadata, and provenance-tracking attributes. Pairwise Weir and Cockerham's FST and Nei's genetic distance estimates aligned with HierFstat expectations based on Pearson correlations and aggregate error metrics. D-statistics conformed to theoretical expectations across eleven simulated datasets spanning a range of introgression signal strengths.

Conclusions: SNPio provides a reproducible Python-native workflow for processing, filtering, encoding, visualizing, and analyzing SNP datasets. It integrates common early-stage population genomic operations into a transparent, scriptable framework, which ultimately promotes workflow provenance and reduces reliance on disjointed software

tools, unsaved terminal commands, and custom scripts. SNPio is particularly suited for population genomic studies of non-model organisms in ecological, evolutionary, and conservation contexts, where reproducible preprocessing and interoperability with downstream analyses are becoming increasingly important.

Keywords: SNP; population genomics; filtering; genotype encoding; bioinformatics software; workflow; FST; genetic distance; D-statistics; introgression

Background

The proliferation of high-throughput sequencing has fundamentally upscaled the analytical complexity of population genomics [1, 2]. Consequently, researchers frequently rely on fragmented workflows constructed by chaining disparate command-line utilities, programming environments, and custom file conversion scripts [3, 4]. This operational fragmentation reduces transparency, complicates computational reproducibility, and introduces a risk of systemic errors during data preprocessing [5]. Although workflow management frameworks such as Nextflow and Snakemake can orchestrate these pipelines [6, 7], the initial development and continuous maintenance of the underlying custom data-wrangling scripts remain substantial bottlenecks [8, 9].

These pipeline inefficiencies are apparent in the population genomics of non-model organisms, where single nucleotide polymorphism (SNP) datasets form the basis for inferring population structure, introgression, and phylogenetic relationships [10–13]. While the software ecosystem for SNP analysis is robust, it is highly partitioned by underlying programming language architectures. Highly optimized C-compiled utilities

like BCFtools [14] excel at rapid variant manipulation but exist outside interactive analytical environments. Conversely, the R ecosystem provides mature, statistically rigorous interfaces for spatial and evolutionary ecology (e.g., snpR [15], vcfR [16], SNPfiltR [17], HierFstat [18], vegan [19]).

Despite Python operating as the prevailing standard for modern numerical computing and machine learning [20, 21], the language has a limited variety of consolidated, population genomics-focused preprocessing toolkits of comparable scope to analogous R software (but see [22–29]). Furthermore, existing software frequently obfuscates the algorithmic assumptions driving data filtering and transformation [30], leaving users disconnected from the intermediate consequences of their preprocessing decisions (e.g., [31, 32]). Given the increasing complexity of modern population genomic workflows, reliance on disconnected Python, C/ C++, and R tools can introduce friction between preprocessing, quality control, statistical analysis, and downstream data export. Accordingly, there is a need for software that unifies early-stage SNP workflow steps while preserving transparency, reproducibility, and export into file or Python object formats accepted by data science libraries (e.g., Scikit-Learn, PyTorch) or population genomic software.

Here, we present SNPio to address this need by providing a cohesive Python-native framework for SNP data parsing, filtering, visualization, summary statistic and introgression analyses, and numerical encoding (Fig. 1; Table 1). SNPio supports common population genomic file formats (Variant Call Format, VCF [33]; PHYLIP [34]; STRUCTURE [35]; GENEPOP [36]), strictly preserves the order of operations during filtering protocols, and exports numerical genotype encodings into formats accepted by

popular Python deep learning and statistical libraries (e.g., NumPy, Scikit-Learn, PyTorch, TensorFlow [37–40]). It also estimates population genomic summary statistics and detects signals of introgression, analyses that are often used in evolutionary, ecological, and conservation contexts. Each module generates diagnostic visualizations and tabular outputs that summarize filtering effects, summary statistics, and analysis results. Importantly, SNPio centralizes file handling, quality control, and exploratory data analysis operations. By doing so, it reduces implementation burden, improves workflow integration, and limits opportunities for inconsistencies introduced by manual format conversion or the cobbling together of disconnected software tools.

Implementation

Software architecture and data model

SNPio is implemented in the Python programming language [41] and structured around an object-oriented genotype container architecture. Format-specific modules parse input files and instantiate a shared "GenotypeData" class (Table 1). This central object encapsulates genotype matrices, sample and locus identifiers, optional population metadata (a population map; [28]), and various other attributes. By standardizing the internal representation of disparate file formats, downstream modules operate independently of the original input structure, facilitating natural format conversion. For VCF input, SNPio utilizes the "pysam" library to extract records and metadata [14, 42], caching the extracted data in HDF5 file structures to balance memory usage with data read and write speed [43]. For VCF input, SNPio retains reference and alternate allele metadata during parsing so downstream genotype encodings preserve VCF allele

semantics. Genotypes are internally represented as NumPy arrays utilizing standard IUPAC nucleotide letter codes [40, 44]. This array-based representation natively accommodates missing and ambiguous (i.e., heterozygous) states while ensuring structural compatibility with downstream SNPio modules.

Filtering and preprocessing

Filtering is executed through the "NRemover2 class" (Table 1), which utilizes a chainable syntax structurally analogous to pandas method chaining [45]. Available operations include per-locus, per-sample, and population-aware per-locus missingness thresholds; removal of monomorphic, singleton, and non-biallelic loci; minimum minor allele frequency (MAF) and minor allele count (MAC) limits; allele-depth filtering for VCF-derived metadata; coordinate-based thinning by physical distance within a scaffold/ chromosome; and randomized locus subsetting. To strictly maintain data provenance, filtering decisions are non-destructive. Instead, SNPio registers retention criteria using Boolean NumPy masks, ensuring that the precise order of operations and the specific loci or samples dropped at each discrete step are traceable. Filtering histories are retained for provenance and reporting rather than as an interactive undo/ redo system; users can branch analyses by copying the genotype object or rerunning the filtering chain from a defined starting state.

Genotype encodings for downstream analysis

SNPio translates genotype matrices into numerical encodings via the "GenotypeEncoder" module (Table 1). Such encodings are used by popular data science frameworks such as Scikit-Learn and PyTorch [37, 38]. Supported transformations include 0/1/2 genotype encodings for biallelic loci, one-hot multi-label nucleotide encodings, ten-integer IUPAC encodings, and a two-channel biallelic representation. For VCF-derived datasets, 0/1/2 encodings preserve VCF reference/ alternate genotype semantics, where 0=homozygous reference, 1=heterozygous, and 2=homozygous alternate genotypes. For non-VCF inputs lacking explicit reference/ alternate metadata, allele states are assigned using documented SNPio allele-coding rules (see Availability of data and materials for a link to the documentation). Missing genotypes are preserved explicitly within the numeric arrays rather than being subjected to internal imputation or outright removal of samples or loci. This purposeful decision promotes data provenance, transparency, and user choice.

Population genetic analyses

SNPio provides native implementations and integrated interfaces for a suite of fundamental population genetic analyses in the "PopGenStatistics" module (Table 1). First, pairwise genetic differentiation is calculated via Weir and Cockerham F_{ST} and Nei's genetic distance [46, 47], supporting optional permutation testing and bootstrap confidence intervals. Second, SNPio computes within-population summary statistics, including observed and expected heterozygosity (H_o, H_e) and π (average number of within-population nucleotide differences; [48]). Third, principal component analysis

(PCA) is available for two and three-dimensional visualization of population structure [49], with per-sample color gradients representing missingness proportion. Under the hood, SNPio utilizes the Scikit-Learn PCA implementation [37]. Fourth, SNPio can execute introgression analyses, including the four-taxon Patterson's D-statistic [50, 51] and five-taxon Partitioned-D [52] and D_{FOIL} tests [53]. The D-statistic modules follow the logic of Comp-D [54], utilizing Numba just-in-time compilation to accelerate computationally intensive routines [55]. These analyses are included to provide an integrated preprocessing-to-analysis workflow rather than to introduce new statistical estimators.

Reporting and outputs

To support computational reproducibility, SNPio automatically generates comprehensive metadata outputs corresponding to each executed workflow. SNPio exports discrete tabular summaries and JavaScript Object Notation (JSON) configuration files that document all user-defined parameters and filtering threshold impacts. Visualizations (e.g., PNG, PDF, JPEG) are generated programmatically to illustrate summary statistics and dataset filtering attrition. Additionally, SNPio integrates directly with MultiQC [56], aggregating executed filtering metrics, statistical outputs, and diagnostic plots into a unified, interactive HTML report. Representative examples of generated visual outputs can be found in the SNPio documentation (see Availability of data and materials).

Computational Resource Benchmarking

To evaluate the computational efficiency of SNPio, execution times and peak memory utilization were quantified against a progression of dataset sizes. Five empirical datasets were randomly subset from Meramveliotakis *et al.* [57], each containing *N*=166 samples and ranging progressively from 1,000 to 1,000,000 loci. Benchmarking was conducted using Hyperfine to measure execution duration [58], while peak memory allocation was independently tracked utilizing custom Python and R scripts (see Open Science Framework repository link in Availability of data and materials). Performance metrics were recorded across 100 replicates for each dataset size to capture statistical stability. To maintain a structurally equivalent comparison of memory-resident analytical environments, SNPio was benchmarked directly against the R packages vcfR for file parsing and SNPfiltR for filtering operations [16, 17]. Stream-based, C-compiled parsers such as BCFtools [14] were excluded from direct performance comparisons because their transient memory architectures are functionally incongruent with the persistent object models required for interactive analytical workflows in Python or R. Python packages that expose htslib- or pysam-backed VCF parsing were not treated as independent performance baselines because such comparisons would largely benchmark shared lower-level parsing backends rather than SNPio's higher-level workflow architecture. Accordingly, the benchmarks were designed to compare SNPio against memory-resident analytical environments used for interactive population genomic workflows, not to establish universal parsing superiority across all available software. Moreover, SNPio is intended for single-machine, memory-resident analyses of reduced-representation and other SNP datasets rather than distributed processing of massive whole-genome datasets.

Validation of Population Genetic Statistics

The analytical accuracy of the SNPio Weir and Cockerham's [46] FST and Nei's [47] genetic distance estimates was compared against established software, HierFstat v0.5-11 [18, 46], using the SNPio example dataset (*N*=175 samples, 14,000 SNP loci; [31]). The HierFstat FST analysis used 1,000 bootstrap replicates and 1,000 permutations to estimate 95% confidence intervals and assess significance, whereas HierFstat only supports estimation of observed Nei's genetic distances. To quantify agreement between SNPio and HierFstat estimates, pairwise population matrices were compared using Mantel tests, with Pearson's correlation coefficients computed with the vegan v2.7-1 R package [19]. We also calculated several aggregated error metrics, including root mean squared error (RMSE), mean absolute error (MAE), maximum absolute error, and mean signed error.

Simulation and Validation of Introgression Metrics

To validate the implementations of Patterson's D [50, 51], Partitioned-D [52], and D_{FOIL} [53] statistics, demographic models were simulated using the Demes v0.2.3 and msprime v1.3.4 Python packages [59, 60]. We generated 22 *in silico* sequence alignments: 11 datasets for four-taxon Patterson's D tests and 11 datasets for five-taxon Partitioned-D and D_{FOIL} tests. The base demographic model specified a constant effective population size (N_e) of 5.0×10^5, with global mutation and recombination rates

fixed at 3.0 x 10^{-8} per base pair per generation across a total sequence length of 1.0 x 10^{6} base pairs. Each defined population contained *N*=20 sampled individuals.

Admixture was modeled as a discrete introgression event from population 3 into population 2 occurring 1,000 generations before present, executed via the "add_pulse" method in Demes. The strength of this introgression pulse was varied across 11 values: 0.01 and 0.1–1.0 in increments of 0.1. Sites with ambiguous outgroup polarization, including sites where the outgroup allele frequency was exactly 0.5, were excluded before D-statistic calculation. To enforce computational reproducibility, the initial random seed was set to "12345" and sequentially incremented by one for each generated dataset. The Demes graph objects were subsequently converted into msprime models to leverage the native VCF export functionality, providing the raw input files for SNPio validation.

Results and Discussion

We evaluated the computational efficiency of SNPio against comparable memory-resident R software [61], comparing file parsing against vcfR and filtering operations against SNPfiltR (Fig. 2; [16, 17]). In the benchmarks reported here, SNPio was evaluated on datasets each containing *N*=166 samples and a range of 1,000 to 1,000,000 loci. Across the six benchmarked dataset sizes (derived from Meramveliotakis *et al.* [57]), SNPio parsed files faster than vcfR for datasets of 50,000 loci or larger, with vcfR requiring 1.57-2.05x longer runtime than SNPio (Fig. 2A; Supplementary Table 1; Supplementary Table 2). For smaller datasets, vcfR was faster, requiring 0.42x and 0.92x the SNPio runtime at 1,000 and 10,000 loci, respectively. vcfR used less memory across

all file parsing benchmarks, with memory use ranging from 0.11-0.81x that of SNPio (Fig. 2B; Supplementary Table 1; Supplementary Table 3). For filtering, SNPio was consistently faster than SNPfiltR across all filtering operations and dataset sizes (Fig. 3; Supplementary Table 1; Supplementary Table 2). SNPfiltR required 6.24-370.90x longer runtime than SNPio, with the largest differences observed for missingness per sample and biallelic filtering. SNPfiltR used less memory than SNPio across filtering benchmarks, with memory use ranging from 0.0004-0.24x that of SNPio (Fig. 4; Supplementary Table 1; Supplementary Table 3). The increased memory utilization is an explicit structural tradeoff: SNPio retains NumPy-backed genotype arrays, metadata, and provenance-tracking objects internally to facilitate interoperability with downstream modules.

To corroborate the accuracy of the implemented population genetic analysis modules, we validated the SNPio estimates against established software or theoretical expectations from simulated datasets. Pairwise F_{ST} and Nei's genetic distance estimates demonstrated parity with HierFstat v0.5-11 [18], yielding perfect or near-perfect matrix correlations and low aggregate error. For F_{ST}, RMSE and MAE were 0.032 and 0.026, respectively, while Nei's genetic distance showed RMSE and MAE values <0.001 (Table 2). Detailed pairwise estimates, SNPio confidence intervals and empirical *P*-values, and SNPio – HierFstat differences of observed statistics are provided for Weir and Cockerham's F_{ST} and Nei's genetic distance in Tables 3 and 4, respectively. Similarly, analysis of simulated alignments with uniformly varied admixture signal strength confirmed that the Patterson's D (Fig. 5A), Partitioned-D (Fig. 5B), and D_{FOIL} (Fig. 5C) statistics conformed to expected theoretical patterns [51–53].

The computational landscape for population genomic analysis of non-model organisms is presently partitioned across distinct operational paradigms. Command-line utilities executed in C/ C++, such as BCFtools [14], are optimized for raw computational speed and low-memory processing of high-density variant data, yet they function entirely outside of interactive analytical environments. Conversely, the R ecosystem has historically dominated statistical ecology and evolutionary biology through packages like vcfR [16], snpR [15], HierFstat [18], and vegan [19]. While these tools provide robust statistical frameworks, they frequently encounter interoperability bottlenecks when connecting multiple software packages. Furthermore, as population genomic workflow complexity increases, relying on isolated command-line routines or R-centric objects can introduce substantial friction. SNPio bridges this methodological gap by shifting SNP parsing, filtering, genotype encoding, and exploratory data analysis into a unified Python-native framework. Thus, SNPio's primary contribution is architectural rather than statistical: It integrates established SNP-processing and population-genetic operations into a reproducible, object-oriented Python workflow that preserves preprocessing provenance.

To address the challenge of provenance tracking, SNPio implements a dual-reporting architecture. Human-readable reporting is achieved through integration with MultiQC [56], which SNPio uses to generate an interactive report containing results for each executed module. Concurrently, SNPio exports all underlying metrics and configuration states as discrete tabular arrays and JavaScript Object Notation (JSON) objects. This programmatic output layer allows for parsing by automated workflow managers such as Nextflow [6]. Standardizing the metadata and quality control artifacts

generated during execution enables computational reproducibility and mitigates the loss of context that can occur across multi-step population genomic workflows.

While current file handling support encompasses standard formats, including VCF, PHYLIP, STRUCTURE, and GENEPOP, the underlying code architecture facilitates the integration of new file types and novel analytical modules. Consequently, the primary developmental trajectories for SNPio involve expanding its analytical repertoire to include, for example, genomic variance partitioning (e.g., analysis of molecular variance, AMOVA [62]) and supporting additional file formats, such as NEXUS and FASTA [63, 64].

Conclusions

SNPio provides a Python-native framework for parsing, filtering, encoding, visualizing, and analyzing SNP datasets. Rather than replacing highly optimized C-compiled utilities for massive whole-genome analyses, SNPio is designed to standardize early-stage population genomic workflows in scriptable, reproducible Python environments. Its main advantages are integrated file handling, transparent filter tracking, genotype encodings, statistical summaries, automated reporting, and interoperability with downstream Python-based analyses. These features make SNPio particularly useful for non-model organism studies in ecology, evolution, and conservation genomics, where reproducible preprocessing and clear data provenance are essential.

Availability and requirements

Project name: SNPio

Project home page: https://github.com/btmartin721/SNPio

Operating system(s): MacOS, Linux

Programming language: Python

Other requirements: Python 3.11 or 3.12 environment; various Python software dependencies (e.g., Pandas, NumPy, SciPy, Scikit-Learn, pysam)

License: GPL-3.0

Any restrictions to use by non-academics: None beyond license terms

List of abbreviations

BCF: Binary call format

FST: Among-population fixation index

HDF5: Hierarchical Data Format version 5

IUPAC: International Union of Pure and Applied Chemistry

JSON: JavaScript object notation

MAC: Minor allele count

MAE: Mean absolute error

MAF: Minor allele frequency

NGS: Next-generation sequencing

PCA: Principal component analysis

RMSE: Root mean squared error

SNP: Single nucleotide polymorphism

VCF: Variant Call Format

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

The analyses reported here used SNPio v1.6.16, Git commit a6be237b08373b7fd1c48bde09e6e278a85ef775, PyPI release v1.6.16, and Docker image digest sha256:ccde7732033580661c58890eb6c5d3d2d0e4c5a3e3f3b22657381c675262274d. The SNPio distribution released at the time of publication can be found on Zenodo (https://doi.org/10.5281/zenodo.20316228). The Zenodo release archives the exact SNPio source distribution used for the analyses reported here. The benchmarking and validation materials supporting the conclusions of this article are available in an Open Science Framework (OSF) repository (https://doi.org/10.17605/OSF.IO/WDQ3F). The empirical datasets analyzed during benchmarking are publicly available through NCBI BioProject PRJNA563121. The SNPio source code is available in the GitHub repository (https://github.com/btmartin721/SNPio), and user documentation is available at https://snpio.readthedocs.io/en/latest/. Versioned software distributions are available

through PyPI (https://pypi.org/project/snpio/), Anaconda (https://anaconda.org/channels/btmartin721/packages/snpio/overview), and DockerHub (https://hub.docker.com/r/btmartin721/snpio).

Competing interests

The authors declare that they have no competing interests.

Funding

This work was supported by the Seton Hall University Research Council (URC).

Authors' contributions

BTM conceived the project, developed software components, performed analyses, and drafted the manuscript. DRM and NS performed analyses, software validation and testing, and contributed to software documentation and manuscript preparation. SMM contributed to conceptual design, validation context, and manuscript preparation. TKC contributed to software implementation and design, benchmarking/ validation planning, and manuscript preparation. All authors read and approved the final manuscript.

Acknowledgements

We thank Z.D. Zbinden for design contributions and feedback. Any use of trade, firm, or product names is for descriptive purposes only and does not imply endorsement by the

U.S. Government. The findings and conclusions are those of the authors and do not necessarily represent the views of the U.S. Fish and Wildlife Service.

Authors' information

BTM is an Assistant Professor at Seton Hall University. DRM and NS are undergraduate research assistants at Seton Hall University. SMM is with the U.S. Fish and Wildlife Service. TKC is with the University of Arkansas.

References

1. Wong K-C. Big data challenges in genome informatics. Biophys Rev. 2019;11:51–4.

2. Schweizer RM, Saarman N, Ramstad KM, Forester BR, Kelley JL, Hand BK, et al. Big Data in Conservation Genomics: Boosting Skills, Hedging Bets, and Staying Current in the Field. J Hered. 2021;112:313–27. https://doi.org/10.1093/jhered/esab019.

3. Djaffardjy M, Marchment G, Sebe C, Blanchet R, Belhajjame K, Gaignard A, et al. Developing and reusing bioinformatics data analysis pipelines using scientific workflow systems. Comput Struct Biotechnol J. 2023;21:2075–85. https://doi.org/10.1016/j.csbj.2023.03.003.

4. Leipzig J. A review of bioinformatic pipeline frameworks. Brief Bioinform. 2017;18:530–6. https://doi.org/10.1093/bib/bbw020.

5. Ziemann M, Poulain P, Bora A. The five pillars of computational reproducibility: bioinformatics and beyond. Brief Bioinform. 2023;24:bbad375. https://doi.org/10.1093/bib/bbad375.

6. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. Nat Biotechnol. 2017;35:316–9. https://doi.org/10.1038/nbt.3820.

7. Köster J, Rahmann S. Snakemake—a scalable bioinformatics workflow engine. Bioinformatics. 2012;28:2520–2. https://doi.org/10.1093/bioinformatics/bts480.

8. Hajieghrari B, Nejati-Jahromi S. Next generation sequencing and beyond: a review of genomic sequencing methods. Funct Integr Genomics. 2025;25:242. https://doi.org/10.1007/s10142-025-01724-9.

9. Ekblom R, Wolf JBW. A field guide to whole-genome sequencing, assembly and annotation. Evol Appl. 2014;7:1026–42. https://doi.org/10.1111/eva.12178.

10. Seeb JE, Carvalho G, Hauser L, Naish K, Roberts S, Seeb LW. Single-nucleotide polymorphism (SNP) discovery and applications of SNP genotyping in nonmodel organisms. Mol Ecol Resour. 2011;11:1–8. https://doi.org/10.1111/j.1755-0998.2010.02979.x.

11. Leaché AD, Oaks JR. The Utility of Single Nucleotide Polymorphism (SNP) Data in Phylogenetics. Annu Rev Ecol Evol Syst. 2017;48:69–84. https://doi.org/10.1146/annurev-ecolsys-110316-022645.

12. Twyford AD, Ennos RA. Next-generation hybridization and introgression. Heredity. 2012;108:179–89. https://doi.org/10.1038/hdy.2011.68.

13. Lou RN, Jacobs A, Wilder AP, Therkildsen NO. A beginner's guide to low-coverage whole genome sequencing for population genomics. Mol Ecol. 2021;30:5966–93. https://doi.org/10.1111/mec.16077.

14. Danecek P, Bonfield JK, Liddle J, Marshall J, Ohan V, Pollard MO, et al. Twelve years of SAMtools and BCFtools. GigaScience. 2021;10:giab008. https://doi.org/10.1093/gigascience/giab008.

15. Hemstrom W, Jones M. snpR: User friendly population genomics for SNP data sets with categorical metadata. Mol Ecol Resour. 2023;23:962–73. https://doi.org/10.1111/1755-0998.13721.

16. Knaus BJ, Grünwald NJ. vcfr: a package to manipulate and visualize variant call format data in R. Mol Ecol Resour. 2017;17:44–53. https://doi.org/10.1111/1755-0998.12549.

17. DeRaad D. snpfiltr: An R package for interactive and reproducible SNP filtering. Mol Ecol Resour. 2022;22:2443–53. https://doi.org/10.1111/1755-0998.13618.

18. Goudet J. hierfstat, a package for r to compute and test hierarchical F-statistics. Mol Ecol Notes. 2005;5:184–6. https://doi.org/10.1111/j.1471-8286.2004.00828.x.

19. Oksanen J, Simpson GL, Blanchet FG, Kindt R, Legendre P, Minchin PR, et al. vegan: Community Ecology Package. 2026. http://CRAN.R-project.org/package=vegan.

20. Ekmekci B, McAnany CE, Mura C. An Introduction to Programming for Bioscientists: A Python-Based Primer. PLOS Comput Biol. 2016;12:e1004867. https://doi.org/10.1371/journal.pcbi.1004867.

21. Hinsen K. High-Level Scientific Programming with Python. In: Sloot PMA, Hoekstra AG, Tan CJK, Dongarra JJ, editors. Computational Science — ICCS 2002. Berlin, Heidelberg: Springer; 2002. p. 691–700. https://doi.org/10.1007/3-540-47789-6_72.

22. De Mita S, Siol M. EggLib 3: A python package for population genetics and genomics. BMC Genet. 2012;13:27. https://doi.org/10.1111/1755-0998.13672.

23. Vitale D, Koretsky MJ, Kuznetsov N, Hong S, Martin J, James M, et al. GenoTools: an open-source Python package for efficient genotype data quality control and analysis. G3 GenesGenomesGenetics. 2025;15:jkae268. https://doi.org/10.1093/g3journal/jkae268.

24. Salek Ardestani S, Mohandesan E. GENOVIS: a Python package for the visualization of population genetic analyses. BMC Genomics. 2026;27:190. https://doi.org/10.1186/s12864-026-12598-x.

25. Lancaster AK, Single RM, Mack SJ, Sochat V, Mariani MP, Webster GD. PyPop: a mature open-source software pipeline for population genomics. Front Immunol. 2024;15. https://doi.org/10.3389/fimmu.2024.1378512.

26. Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. Bioinformatics. 2009;25:1422–3. https://doi.org/10.1093/bioinformatics/btp163.

27. Rand KD, Grytten I, Pavlović M, Kanduri C, Sandve GK. BioNumPy: array programming for biology. Nat Methods. 2024;21:2198–9. https://doi.org/10.1038/s41592-024-02483-4.

28. Catchen J, Hohenlohe PA, Bassham S, Amores A, Cresko WA. Stacks: an analysis tool set for population genomics. Mol Ecol. 2013;22:3124–40. https://doi.org/10.1111/mec.12354.

29. Eaton DAR, Overcast I. ipyrad: Interactive assembly and analysis of RADseq datasets. Bioinformatics. 2020;36:2592–4. https://doi.org/10.1093/bioinformatics/btz966.

30. Hu B, Canon S, Eloe-Fadrosh EA, Anubhav, Babinski M, Corilo Y, et al. Challenges in Bioinformatics Workflows for Processing Microbiome Omics Data at Scale. Front Bioinforma. 2022;1. https://doi.org/10.3389/fbinf.2021.826370.

31. Martin BT, Chafin TK, Douglas MR, Placyk Jr. JS, Birkhead RD, Phillips CA, et al. The choices we make and the impacts they have: Machine learning and species delimitation in North American box turtles (*Terrapene* spp.). Mol Ecol Resour. 2021;21:2801–17. https://doi.org/10.1111/1755-0998.13350.

32. Linck E, Battey CJ. Minor allele frequency thresholds strongly affect population structure inference with genomic data sets. Mol Ecol Resour. 2019;19:639–47. https://doi.org/10.1111/1755-0998.12995.

33. Danecek P, Auton A, Abecasis G, Albers CA, Banks E, DePristo MA, et al. The variant call format and VCFtools. Bioinformatics. 2011;27:2156–8. https://doi.org/10.1093/bioinformatics/btr330.

34. Felsenstein J. PHYLIP-Phylogeny inference package (Version 3.2). Cladistics. 1989;5:164–6. https://ci.nii.ac.jp/naid/10003763325.

35. Pritchard JK, Stephens M, Donnelly P. Inference of Population Structure Using Multilocus Genotype Data. Genetics. 2000;155:945–59. https://doi.org/10.1093/genetics/155.2.945.

36. Rousset F. genepop'007: a complete re-implementation of the genepop software for Windows and Linux. Mol Ecol Resour. 2008;8:103–6. https://doi.org/10.1111/j.1471-8286.2007.01931.x.

37. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: Machine Learning in Python. J Mach Learn Res. 2011;12:2825–30. http://jmlr.org/papers/v12/pedregosa11a.html.

38. Ansel J, others. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In: 29th ACM International Conference on

Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24). 2024. https://doi.org/10.1145/3620665.3640366.

39. Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. 2015. https://dl.acm.org/doi/10.5555/3026877.3026899.

40. Harris CR, Millman KJ, Walt SJ van der, Gommers R, Virtanen P, Cournapeau D, et al. Array programming with NumPy. Nature. 2020;585:357–62. https://doi.org/10.1038/s41586-020-2649-2.

41. Python Software Foundation. Python Language Reference, version 3.12. 2026. https://www.python.org/.

42. Bonfield JK, Marshall J, Danecek P, Li H, Ohan V, Whitwham A, et al. HTSlib: C library for reading/writing high-throughput sequencing data. GigaScience. 2021;10:giab007. https://doi.org/10.1093/gigascience/giab007.

43. Collette A. *Python and HDF5*. O'Reilly Media, Inc; 2013. https://www.oreilly.com/library/view/python-and-hdf5/9781491944981.

44. Cornish-Bowden A. Nomenclature for incompletely specified bases in nucleic acid sequences: recommendations 1984. Nucleic Acids Res. 1985;13:3021–30. https://doi.org/10.1093/nar/13.9.3021.

45. Pandas Development Team. pandas-dev/pandas: Pandas. 2020. https://doi.org/10.5281/zenodo.3509134.

46. Weir BS, Cockerham CC. Estimating F-Statistics for the Analysis of Population Structure. Evolution. 1984;38:1358–70. https://doi.org/10.2307/2408641.

47. Nei M. Genetic Distance between Populations. Am Nat. 1972;106:283–92. https://doi.org/10.1086/282771.

48. Nei M, Li WH. Mathematical model for studying genetic variation in terms of restriction endonucleases. Proc Natl Acad Sci. 1979;76:5269–73. https://doi.org/10.1073/pnas.76.10.5269.

49. Lee C, Abdool A, Huang C-H. PCA-based population structure inference with generic clustering algorithms. BMC Bioinformatics. 2009;10:S73. https://doi.org/10.1186/1471-2105-10-S1-S73.

50. Green RE, Krause J, Briggs AW, Maricic T, Stenzel U, Kircher M, et al. A Draft Sequence of the Neandertal Genome. Science. 2010;328:710–22. https://doi.org/10.1126/science.1188021.

51. Durand EY, Patterson N, Reich D, Slatkin M. Testing for Ancient Admixture between Closely Related Populations. Mol Biol Evol. 2011;28:2239–52. https://doi.org/10.1093/molbev/msr048.

52. Eaton DAR, Ree RH. Inferring Phylogeny and Introgression using RADseq Data: An Example from Flowering Plants (Pedicularis: Orobanchaceae). Syst Biol. 2013;62:689–706. https://doi.org/10.1093/sysbio/syt032.

53. Pease JB, Hahn MW. Detection and Polarization of Introgression in a Five-Taxon Phylogeny. Syst Biol. 2015;64:651–62. https://doi.org/10.1093/sysbio/syv023.

54. Mussmann SM, Douglas MR, Bangs MR, Douglas ME. Comp-D: a program for comprehensive computation of D-statistics and population summaries of reticulated evolution. Conserv Genet Resour. 2020;12:263–7. https://doi.org/10.1007/s12686-019-01087-x.

55. Lam SK, Pitrou A, Seibert S. Numba: a LLVM-based Python JIT compiler. In: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC. New York, NY, USA: Association for Computing Machinery; 2015. p. 1–6. https://doi.org/10.1145/2833157.2833162.

56. Ewels P, Magnusson M, Lundin S, Käller M. MultiQC: summarize analysis results for multiple tools and samples in a single report. Bioinformatics. 2016;32:3047–8. https://doi.org/10.1093/bioinformatics/btw354.

57. Meramveliotakis E, Ortego J, Anastasiou I, Vogler AP, Papadopoulou A. Habitat Association Predicts Population Connectivity and Persistence in Flightless Beetles: A Population Genomics Approach Within a Dynamic Archipelago. Mol Ecol. 2024;33:e17577. https://doi.org/10.1111/mec.17577.

58. Peter D. hyperfine v1.20.0. 2023. Accessed 06-March-2026. https://github.com/sharkdp/hyperfine.

59. Gower G, Ragsdale AP, Bisschop G, Gutenkunst RN, Hartfield M, Noskova E, et al. Demes: a standard format for demographic models. Genetics. 2022;222:iyac131. https://doi.org/10.1093/genetics/iyac131.

60. Baumdicker F, Bisschop G, Goldstein D, Gower G, Ragsdale AP, Tsambos G, et al. Efficient ancestry and mutation simulation with msprime 1.0. Genetics. 2022;220:iyab229. https://doi.org/10.1093/genetics/iyab229.

61. R Core Team. R language definition. Vienna Austria R Found Stat Comput. 2000;3:116. https://cran.r-project.org/doc/manuals/r-release/R-lang.html.

62. Excoffier L, Smouse PE, Quattro JM. Analysis of molecular variance inferred from metric distances among DNA haplotypes: application to human mitochondrial DNA restriction data. Genetics. 1992;131:479-91. https://doi.org/10.1093/genetics/131.2.479.

63. Pearson WR, Lipman DJ, Improved tools for biological sequence comparison. Proc. Natl. Acad. Sci. U.S.A. 1988;85(8):2444-48. https://doi.org/10.1073/pnas.85.8.2444.

64. Maddison DR, Swofford DL, Maddison WP. Nexus: An extensible file format for systematic information. Systematic Biology. 1997;46(4):590-621. https://doi.org/10.1093/sysbio/46.4.590.

Table 1: SNPio feature descriptions. Core SNPio features with descriptions.

Category	Feature	Description
File parsing	Interoperable file input, export, and conversion.	Supported file types: VCF, STRUCTURE, PHYLIP, GENEPOP.
Filtering	Missing Data Filters	Per-locus, per-locus-per-population, and per-sample filters to handle missing data.
	Frequency-based Filtering	Filter SNPs by minor allele frequency (MAF), minor allele count (MAC), singleton presence, removal of monomorphic loci, and biallelic requirements.
	Thinning, Subsetting	Coordinate-based thinning and random loci subsetting.
	Allele Depth	Enforce a minimum allele depth (requires VCF input).
	Threshold Search	Perform a threshold search for each filtering method. Custom filtering orders and thresholds are supported.
Encodings	0,1,2	Encodes homozygous reference, heterozygous, homozygous alternate, and missing genotypes as 0, 1, 2, and -9.
	One-Hot	Encodes alleles in a binary format (e.g., A/A=[1,0,0,0], T/T=[0,1,0,0], A/T=[1,1,0,0], N/N=[0,0,0,0]).
	Integer	Encodes IUPAC nucleotides as integers (0-9) for homozygous and biallelic codes; missing genotypes are encoded as -9.
	Alleles (Two-channel)	Encodes biallelic genotypes into two NumPy matrices (i.e., channels; one for each allele). Per-channel alleles are encoded as 0 (A), 1 (T), 2 (C), 3 (G), and -1 (missing).
Analyses	Allelic Variability and Differentiation	Measures frequency-based genetic variation, heterozygosity, site-wise differentiation.
	Population Structure	Computes pairwise Weir and Cockerham's FST, Nei's genetic distance, Principal Component Analysis (PCA).
	Introgression	D-statistics: Patterson's four-taxon, Partitioned-D, DFOIL.
	Within-population	Observed (Ho) and Expected (He) heterozygosity, π
Summaries	MultiQC Report	An HTML report that contains a multitude of interactive tables and data visualizations and can be loaded locally.
	Individualized Files	CSV, JSON, and text file outputs, as well as stand-alone, publication-ready plots that can be saved in image and vector formats (e.g., PNG, PDF, SVG).

Table 2. Concordance between SNPio and HierFstat estimates of pairwise population differentiation and Nei's genetic distance. Agreement between SNPio and HierFstat was evaluated across all pairwise comparisons among five populations for Weir and Cockerham's [46] F_{ST} and Nei's [47] genetic distance. Pearson correlation and Mantel Pearson's *r* quantify matrix-level concordance between implementations, while RMSE, MAE, maximum absolute error, and mean signed error summarize the magnitude and direction of numerical differences. Mantel *P*-values were obtained using exact permutation testing ($n!$ = 120 permutations, where n = number of populations) across the pairwise distance matrices.

Metric	Weir and Cockerham's F_{ST}	Nei's genetic distance
Pearson correlation	1.000	1.000
Mantel Pearson's r	1.000	1.000
Mantel *P*-value	0.008	0.008
RMSE	0.032	<0.001
MAE	0.026	<0.001
Maximum absolute error	0.053	<0.001
Mean signed error	0.026	<0.001

Table 3: Comparing SNPio and HierFstat Weir and Cockerham's FST estimates.

Upper-triangle cells report SNPio pairwise FST point estimates for a population genomic dataset containing *N*=175 samples and 14,000 SNP loci [31]. Values below each point estimate show Δ = FST(HierFstat) − FST(SNPio). Lower-triangle cells report corresponding empirical *P*-values and 95% confidence intervals [lower, upper] for the SNPio pairwise estimates via 1,000 bootstrap replicates. Asterisks denote statistical significance based on 1,000 permutations: ****P* < 0.001.

	P1	**P2**	**P3**	**P4**	**P5**
P1	-	0.074 Δ = -0.002	0.175 Δ = -0.001	0.463 Δ = -0.021	0.580 Δ = -0.041
P2	<0.001*** 0.064, 0.084	-	0.166 Δ = -0.004	0.482 Δ = -0.027	0.601 Δ = -0.044
P3	<0.001*** 0.161, 0.191	<0.001*** 0.151, 0.181	-	0.459 Δ = -0.024	0.579 Δ = -0.042
P4	<0.001*** 0.441, 0.483	<0.001*** 0.461, 0.502	<0.001*** 0.436, 0.480	-	0.672 Δ = -0.053
P5	<0.001*** 0.563, 0.598	<0.001*** 0.583, 0.617	<0.001*** 0.562, 0.597	<0.001*** 0.654, 0.689	-

Table 4: Comparing SNPio and HierFstat genetic distance estimates. Upper-triangle cells report SNPio pairwise Nei's [47] genetic distance estimates for a population genomic dataset containing *N*=175 samples and 14,000 SNP loci [31]. Values below each point estimate show Δ = NeiD(HierFstat) − NeiD(SNPio). Lower-triangle cells report corresponding empirical *P*-values and 95% confidence intervals [lower, upper] for the SNPio pairwise estimates, estimated using 1,000 bootstrap replicates. Asterisks denote statistical significance based on 1,000 permutations: ****P* < 0.001.

	P1	**P2**	**P3**	**P4**	**P5**
P1	-	0.004 Δ = 0.000	0.01 Δ = 0.000	0.033 Δ = 0.000	0.06 Δ = 0.000
P2	<0.001*** 0.003, 0.004	-	0.009 Δ = 0.000	0.033 Δ = 0.000	0.061 Δ = -0.001
P3	<0.001*** 0.009, 0.011	<0.001*** 0.008, 0.010	-	0.032 Δ = 0.000	0.065 Δ = -0.001
P4	<0.001*** 0.030, 0.035	<0.001*** 0.031, 0.036	<0.001*** 0.030, 0.035	-	0.075 Δ = -0.001
P5	<0.001*** 0.057, 0.064	<0.001*** 0.057, 0.065	<0.001*** 0.061, 0.069	<0.001*** 0.071, 0.080	-

Figure 1: A typical SNPio workflow. Core SNPio modules (outer boxes) and functionalities (inner boxes) ordered as a typical workflow. See Table 1 for per-module descriptions.

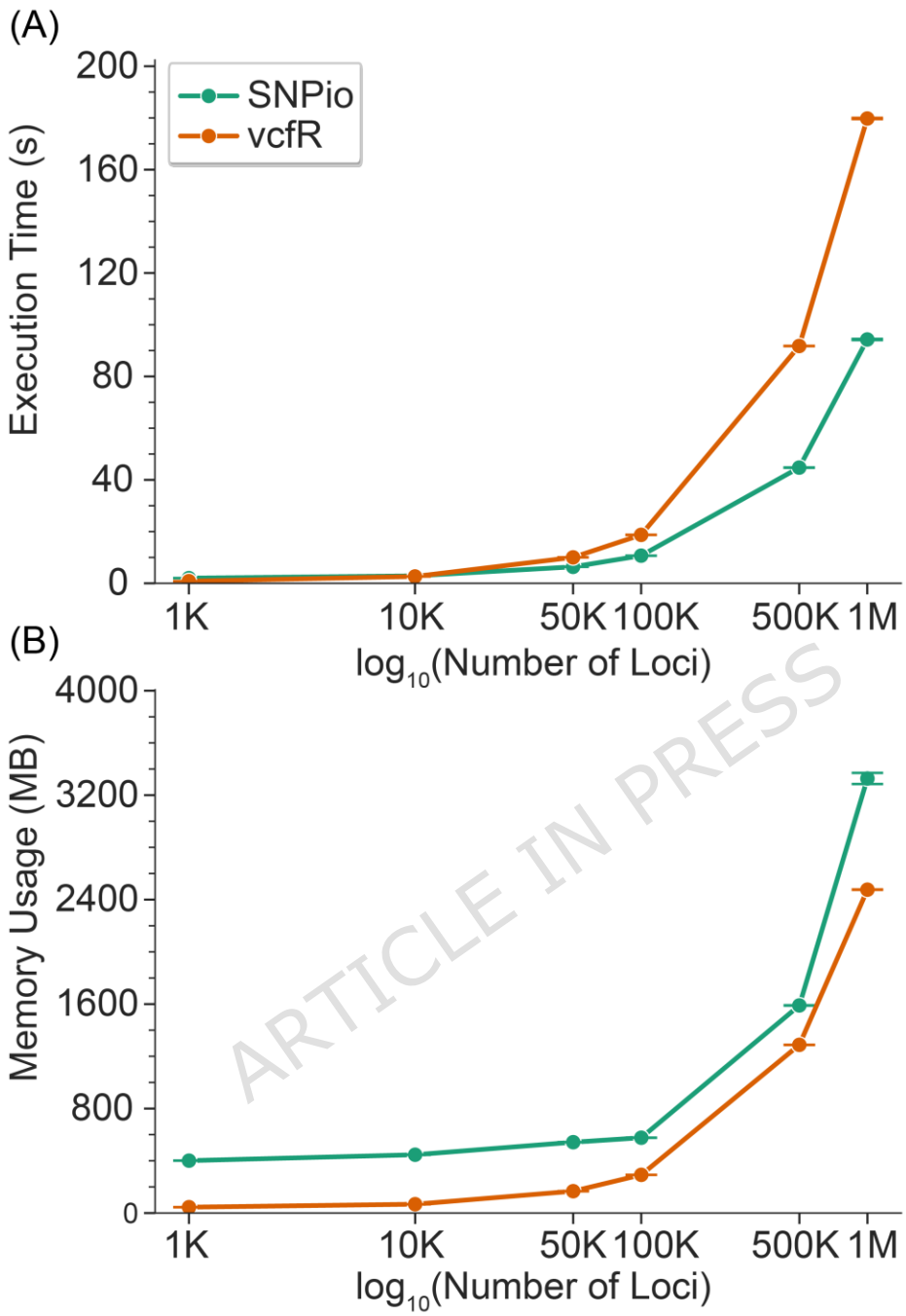

Figure 2: SNPio *versus* vcfR benchmarking comparison. Benchmarks for SNPio and vcfR file parsing include (A) execution time (s=seconds) and (B) memory usage (MB=Megabytes). The benchmark dataset includes *N*=166 samples across a range of SNP alignment lengths. The x-axis is log10-scaled; K = thousand and M = million loci. Error bars depict 95% confidence intervals across 100 replicates.

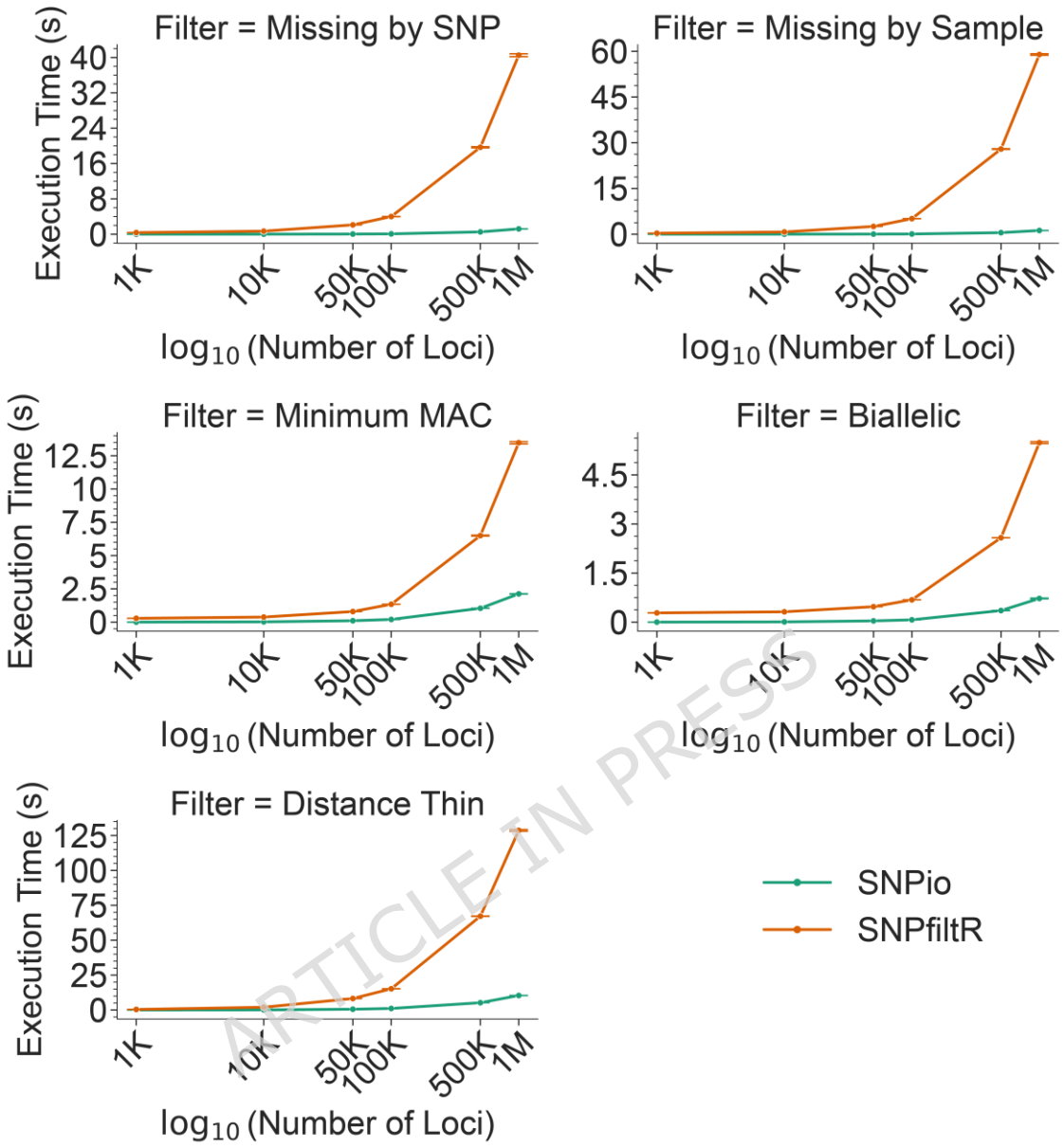

Figure 3: SNPio *versus* SNPfiltR filtering execution time benchmarks. Execution time benchmarks for five filtering operations: per-locus and per-sample maximum missingness thresholds of 50%, minor allele count (MAC) < 2, biallelic locus filtering, and coordinate-based thinning within 100 base pairs on each scaffold/chromosome. Each benchmark dataset includes *N*=166 samples across a range of SNP alignment lengths from 1,000 to 1,000,000 loci. The x-axis is log10-scaled; K = thousand and M = million loci. Error bars represent 95% confidence intervals among 100 replicates.

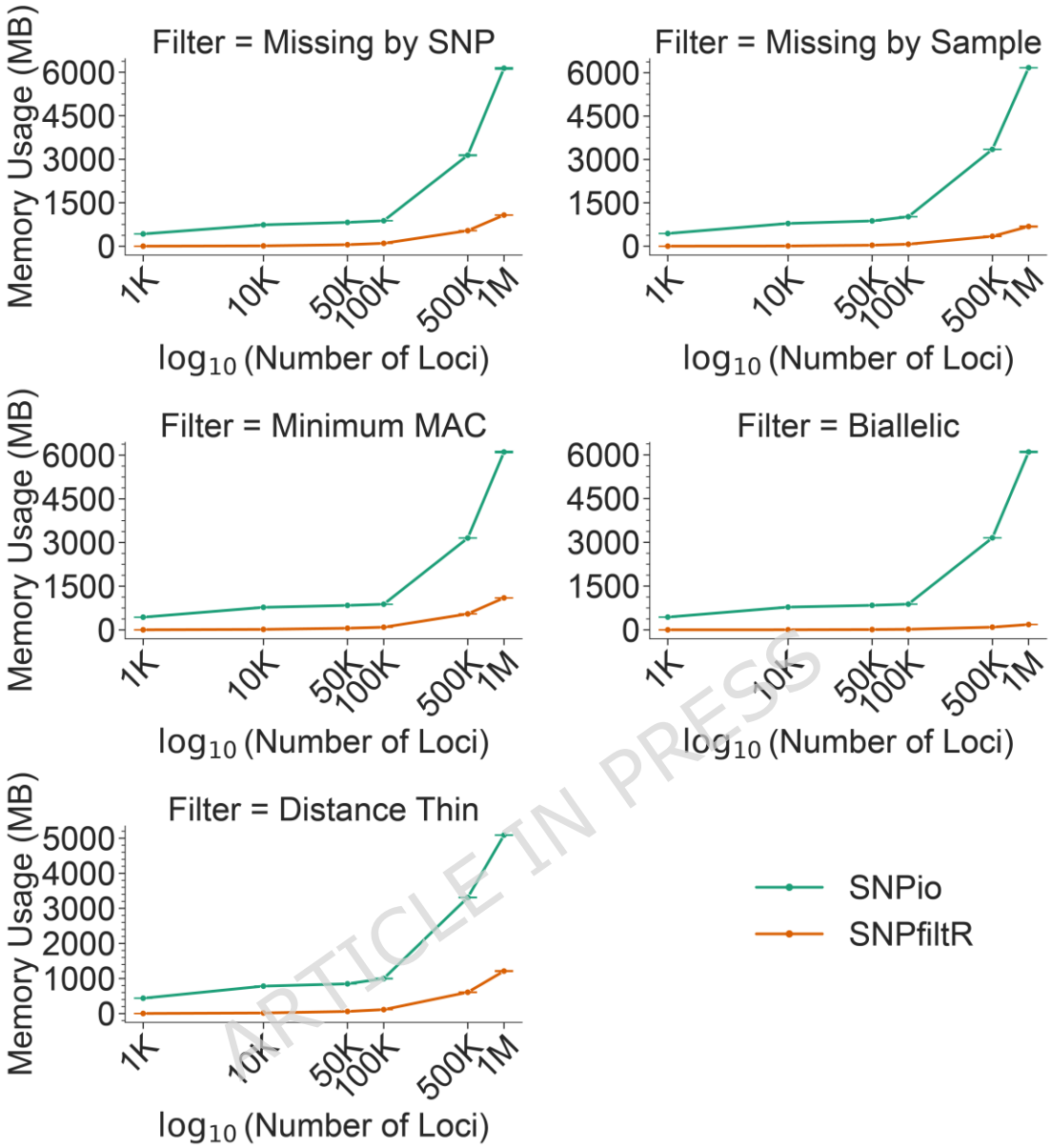

Figure 4: SNPio *versus* SNPfiltR filtering memory usage benchmarks. Memory usage benchmarks for five filtering operations: per-locus and per-sample maximum missingness thresholds of 50%, minor allele count (MAC) < 2, biallelic locus filtering, and coordinate-based thinning within 100 base pairs on each scaffold/chromosome. Each benchmark dataset includes N=166 samples across a range of SNP alignment lengths from 1,000 to 1,000,000 loci. The x-axis is log10-scaled; K = thousand and M = million loci. Error bars represent 95% confidence intervals among 100 replicates.

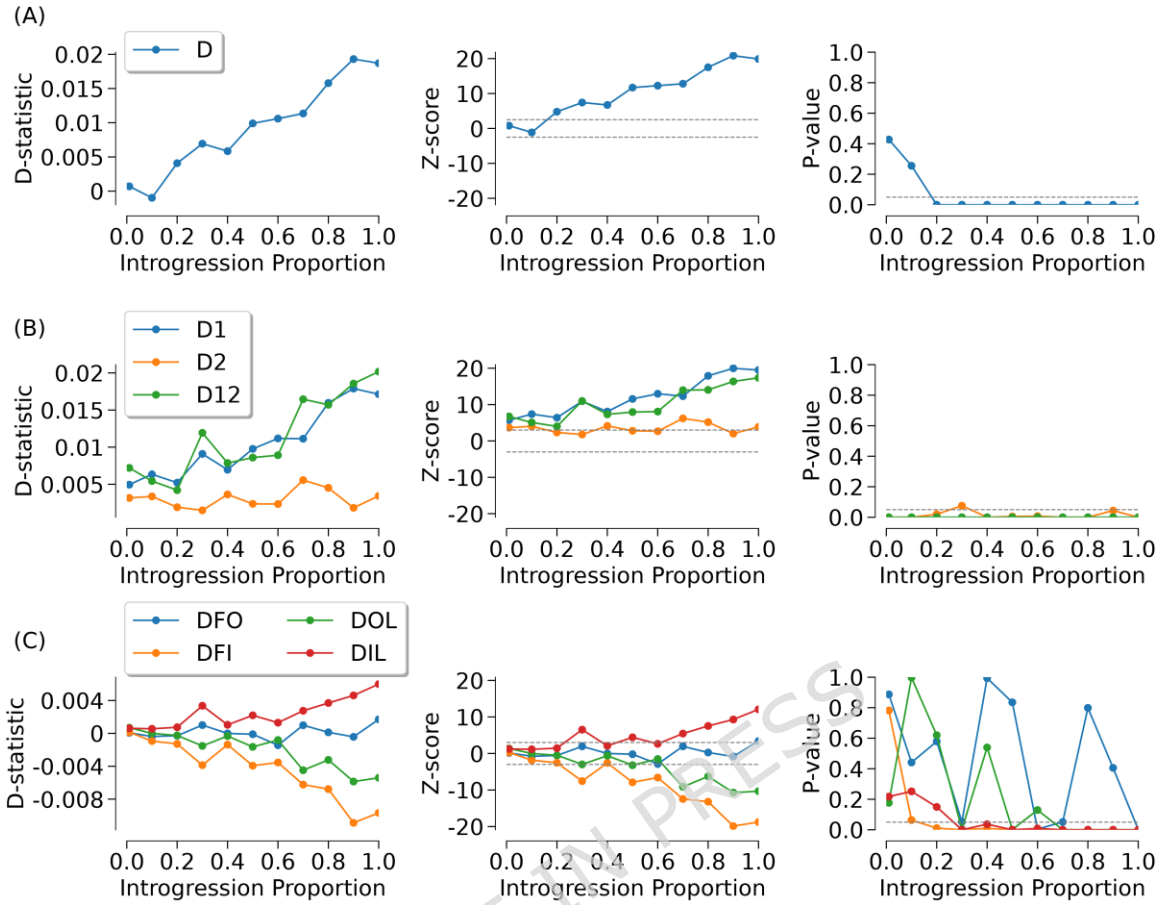

Figure 5: D-statistic validation analysis. SNPio D-statistic validation against eleven simulated migration pulses (*N*=20 samples per population), simulated 1,000 generations before present. Each column of panels shows: (left) Means of observed D-statistics, (middle) Z-scores, and (right) *P*-values. Rows depict (A) Patterson's four-taxon D-statistic test; (B) the five-taxon Partitioned-D statistics, D1, D2, and D12; and (C) the D_{FOIL} statistics (D_{FO}, D_{FI}, D_{OL}, and D_{IL}). Dashed gray lines in the Z-score (-2.5 to 2.5) and *P*-value (α = 0.05) panels indicate common significance thresholds.